1 Introducing the Ford-Fulkerson Algorithm

Problem: Given a directed graph $G = (V, E)$, and a **positive integer** capacity c_e for each edge, along with distinguished vertices s (the "source") and t (the "sink"), and this definition of "flow":

Definition An $s - t$ flow puts f_e flow on each edge e such that:

- Each capacity is obeyed; this is called the "capacity constraint"

$$0 \leq f_e \leq c_e$$

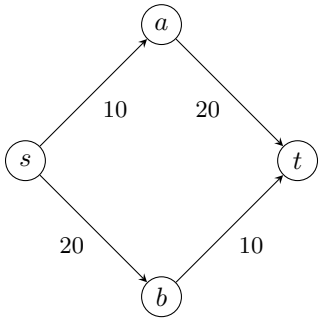
- "Conservation of flow": only the source can create flow, and only the sink can consume it. More formally, for each $v \neq s, t$:

$$\sum_{e \text{ into } v} f_e = \sum_{e \text{ out of } v} f_e$$

- The source can create any amount of flow, and the sink can consume any amount. We say the *value* of a given flow f is:

$$v(f) = \sum_{e \text{ out of } s} f_e$$

Our goal is to find a valid flow with maximum possible $v(f)$, the "maximum flow."

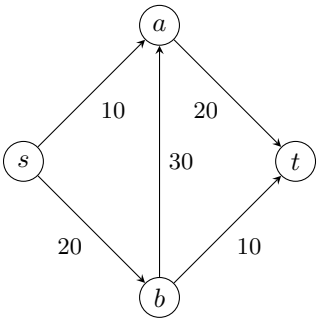


Let's first try a basic greedy algorithm: as long as we can add flow, do so.

$\forall_e \;\; f_e = 0$
while $\exists$ path p from s to t in G with all $c_e > 0$ **do**
  p = any simple s to t path in G, using only edges with $c_e > 0$
  b = min capacity edge on p
  for all edges e in p **do**
    $f_e = f_e + b$
    $c_e = c_e - b$
for all edges e **do**
  Final flow along e is f_e

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

Let's try it with another graph and see if it works:



What we end up needing is a way to "take back" flow that we put on an edge.

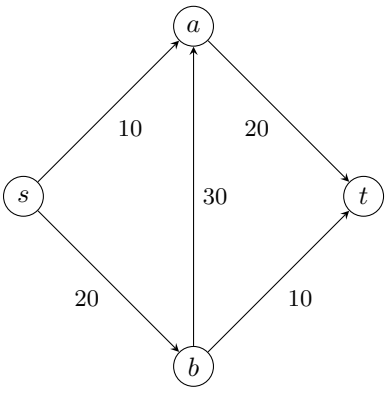
We will define a *residual graph* G_f, for any given flow f, where the capacities on each edge are :

- If $e = (u, v) \in G$, then c_e in G_f is $c_e - f_e$. In other words, the capacity is the remaining capacity that has yet to be used by the flow. We call these **forward** edges.
- If $e = (u, v) \in G$, define $e' = (v, u)$. $c_{e'}$ in G_f is f_e. In other words, the capacity of e' is the flow that can be "taken back" – that was tentatively sent from u to v. We call these **backward** edges.

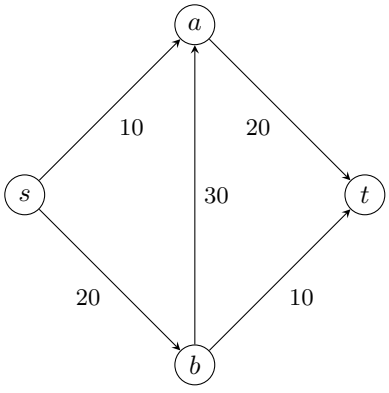
The changes we make will result in this algorithm, known as FORD-FULKERSON.

$\forall_e \quad f_e = 0$
while $\exists$ path p from s to t in G_f **do**
  p = any simple s to t path in G_f.
  b = min residual capacity edge on p (the "bottleneck" edge)
  for all edges $e = (u, v)$ in p **do**
    if e is forward **then**
      $f_e = f_e + b$
    else
      $e' = (v, u)$
      $f_{e'} = f_{e'} - b$

Graph (mark flow on this one)



Residual graph



© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

Let's talk a little bit about this algorithm:

Question 1. Recall that all c_e values are positive integers. Will any f_e values ever be non-integer? Why or why not?

Question 2. What is the minimum increase in $v(f)$ each iteration of the while loop?

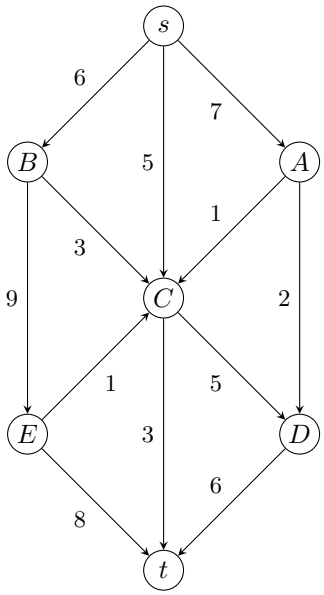
Question 3. Define $C = \sum c_e$ over all $e = (s, v)$. How many iterations will the **while** loop take in the worst case?

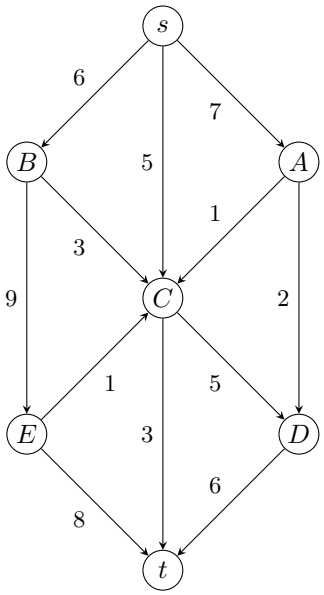
Question 4. How long does each iteration of the **while** loop take?

Question 5. What is the total running time of the algorithm?

Question 6. Is this polynomial time?
Recall that polynomial time is a formalism that fits our intuition for an algorithm that scales efficiently as the input size doubles. Suppose the integer variables for all capacities is a C++ ***int*** *(32 bits) and then we change it to allow for* ***long****s (64 bits) instead, thus doubling the amount of space used to represent edge capacities. What effect does this have, if any, on the variables in the running time found in the previous question?*

Question 7. Find the maximum flow in the following graph. What is the total value of the flow?





© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

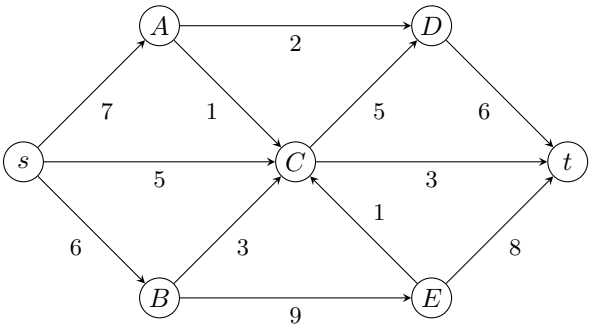
2 Ford-Fulkerson Proof of Correctness

Let's prove that the Ford-Fulkerson algorithm actually finds the maximum flow. To do this, we need one more definition:

Define (A,B) cut: a partition of V into sets A and B such that $s \in A$, $t \in B$. We say that the *capacity* of the cut is:

$$c(A, B) = \sum_{e \text{ out of } A} c_e$$

Find the **min cut** in the following graph. This is the same graph on which we found maximum flow recently:



The following facts will be useful to us in proving that it is a max flow; each of these is non-obvious, and therefore requires a proof.

1. If f is any $s-t$ flow and (A, B) is any $s-t$ cut, then $v(f) = f^{\text{out}}(A) - f^{\text{in}}(A)$. This shouldn't surprise you: intuitively, the total that flows from s to t is the net amount that exits from set A.

2. If f is any $s-t$ flow and (A, B) is any $s-t$ cut, then $v(f) = f^{\text{in}}(B) - f^{\text{out}}(B)$.

3. If f is any $s-t$ flow and (A, B) is any $s-t$ cut, then $v(f) \leq c(A, B)$

Once we have those facts proven, we can show that the algorithm does find a maximum flow. We will show a valid flow and a cut of equal value, and it follows that the flow is a maximum and the cut is a minimum.

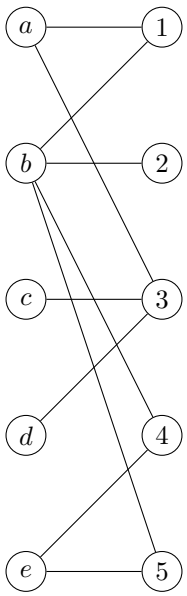
In particular, the claim is as follows. If there is no $s-t$ path in G_f, then there is a cut (A^*, B^*) such that $v(f) = c(A^*, B^*)$.

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

3 Bipartite Matching: an application of network flow

The **maximum bipartite matching** problem is as follows. We are given a **bipartite** graph as input. Our **output** is the maximum number of edges such that no *vertex* is included in more than one edge that we output.

Question 8. What is the maximum bipartite matching in the following graph? This is two questions: what is the maximum number of edges you can select, and also which edges?



Question 9. How can we use maximum flow to find the maximum bipartite matching in a bipartite graph?

Question 10. Suppose we have an input bipartite graph $G = (V, E)$ and wish to find a maximum bipartite matching. How long does the algorithm take?

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

Recall the following problem from the greedy algorithms homework:
Question 11. We're asked to help the captain of the UCI's tennis team to arrange a series of matches against Long Beach State's team. Both teams have n players; the tennis rating (a positive number, where a higher number can be interpreted to mean a better player) of the ith member of UCI's team is a_i and the tennis rating for the kth member of LBSU's team is s_k. We would like to set up a competition in which each person plays one match against a player from the opposite school. Because we get to select who plays against whom, our goal is to make sure that in *as many matches as possible*, the UCI player has a higher tennis rating than their opponent.

The homework question asked you to find a greedy algorithm to optimize the number of matches. Today, we will use network flow to find this.

1. Let's use network flow to solve this problem.

   (a) Define a graph G that we will use in future steps. What property in the graph are we looking for?

   (b) Create a source and sink for G; also, add edge capacities.

   (c) Suppose we find max flow on G. What does $v(f^*)$ tell us about the solution to the problem? How can we use it to prepare the matches?

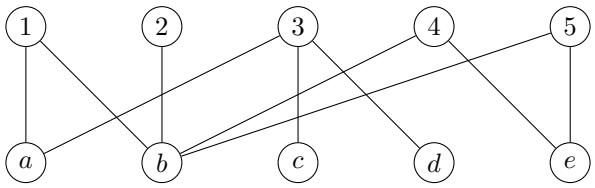
2. Suppose it is possible to win every match. It should be very easy to use $v(f^*)$ to convince someone of this fact. How?

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

3. Suppose it isn't possible, and at least one match will be lost. Someone who hasn't studied network flow (or greedy algorithms, or taken CompSci 161) probably won't accept a "proof" of this fact that amounts to "my algorithm returned a less than perfect match."
   How could we use a similar algorithm such that the output can convince such a person of our unfortunate situation?

To answer this, we're going to use **Hall's Theorem**. For any $A \subseteq V$, we define $\Gamma(A)$ (the "neighborhood" of A) to be the set of vertices that are adjacent to one or more elements of A. If we have a bipartite graph with partition $L \cup R$, where $|L| = |R|$, then either the graph has a perfect matching or there is an $A \subseteq V$ such that $|\Gamma(A)| < |A|$.

For example, prove that the following bipartite graph *does not* have a perfect matching:



If our graph from the tennis problem *does not* have a perfect matching, how can we use the result from running network flow to output a proof of this?

Question 12. What is the running time of our algorithm that determines the maximum number of tennis matches we can win, using network flow?

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

4 Maximum Flow in Polynomial Running time

Question 13. Suppose I am going to run Ford-Fulkerson. Recall that the path from $s - t$ in the residual graph that is augmented is not specified. I want to select a "good" path if one exists. How can I find out if a path with residual capacity *at least* Δ (for some given integer Δ) remains in G_f?

We previously saw an algorithm (Ford-Fulkerson) with running time $O(Cm)$, which we also saw was not polynomial time. Let's modify the algorithm for it to be polynomial time.

$\forall_e \;\; f_e = 0$
$X =$ maximum c_e out of s
for $\Delta = 2^{\lfloor \log_2 X \rfloor}$; $\Delta \geq 1$; $\Delta = \Delta/2$ **do**
  while $\exists$ path p from s to t in $G_f(\Delta)$ **do**
    $p =$ any simple s to t path in $G_f(\Delta)$
    $b =$ min residual capacity edge on p
    for all edges $e = (u, v)$ in p **do**
      if e is forward **then**
        $f_e = f_e + b$
      else
        $e' = (v, u)$
        $f_{e'} = f_{e'} - b$

What is that algorithm's running time?

- How many iterations of the outer loop?
- How long will each iteration of the inner **while** loop take?
- How many iterations of the inner **while** loop will happen during each iteration of the outer loop?

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

5 Edge-Disjoint Paths: an application of network flow

This problem statement comes from the Algorithm Design textbook of Kleinberg and Tardos.

We define the *Escape Problem* as follows. We are given a directed graph $G = (V, E)$ (picture a network of roads). A certain collection of nodes $X \subset V$ are designated as *populated nodes*, and a certain other collection $S \subset V$ are designated as *safe nodes*, with X and S being disjoint. In case of an emergency, we want evacuation routes from the populated nodes to the safe nodes. A set of evacuation routes is defined as a set of paths in G so that (i) each node in X is the tail of one path, (ii) the last node on each path lies in S, and (iii) the paths do not share any edges. Such a set of paths gives a way for the occupants of the populated nodes to "escape" to S, without overly congesting any edge in G.

Question 14. Given G, X, and S, show how to decide in polynomial time whether such a set of evacuation routes exists.

Question 15. Suppose we have exactly the same problem as in the previous part, but we want to enforce an even stronger version of the "no congestion" condition (iii). Thus we change (iii) to say "the paths do not share any *nodes*."

With this new condition, show how to decide in polynomial time whether such a set of evacuation routes exists.

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

6 Extensions to the Maximum Flow Problem

In this section, we will look at two generalizations of maximum flow.

6.1 Supply and Demand on Vertices

Suppose we have a variant of the network flow problem. Instead of having a single source (with the ability to generate infinite outgoing flow) and a single sink (with the ability to consume infinite incoming flow), we instead of a set S of sources generating flow and a set T of sinks that consume it. As before, we have integer capacities on every edge.

Unlike the problem we have been dealing with so far, though, source vertices now have a fixed *supply* value and the sinks have fixed *demand*. All units of flow are interchangeable. We are no longer trying to maximize a flow, but rather satisfy all demand using the existing supply.

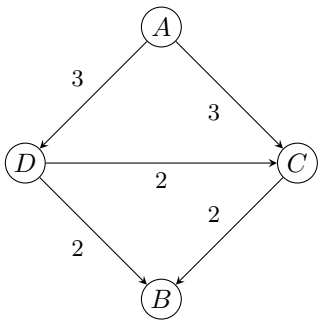
Our **input** is now a directed graph, as before, with integer capacities on the edges. Associated with each **vertex** is a *demand value* d_v. If $d_v > 0$, this indicates a demand, and if $d_v < 0$, it indicates a supply. Vertices with $d_v = 0$ have neither.

All capacities and demands are integer. S is the set of vertices with $d_v < 0$ and T is the set of vertices with $d_v > 0$. The *conservation of flow rule* now changes: supply vertices can now send out $|d_v|$ more flow than they take in, with a similar change for demand vertices.

We can think of this as having edges that are highways or railways and factories (supply) are shipping to outlet stores (demand). All units of flow are of the same type of good.

There is a similar problem we **are not** discussing, called *Multi-commodity flow*. In that problem, different sources have different types of flow, and sinks have different demand types. That one is a much harder problem and we will not discuss it in this class.

For example, consider the following flow network, along with supplies and demands. Note that we no longer have a source or sink in this graph; origins and destinations of flow are handled by supply and demand. Recall that a "negative demand" is supply. In general, a vertex does not need to have a supply or a demand (it can have neither).



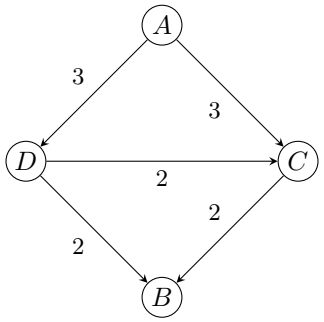
Vertex	Demand
A	-3
B	4
C	2
D	-3

When our input is a graph with supply and demand, we now have a binary question: can all of the demand be filled by the supply at hand? There is a follow-up: if so, how do we route it?

Question 16. For the graph above, *can* the six units of demand be satisfied by the six units of supply? If so, how should the flow be routed?

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

Question 17. How do we design an algorithm to take such a graph (and the supply and demand values) as input and determine if we can do this? *Hint: do not reinvent the wheel.*



Note that this completion means we now have, in effect, a library function for network flow with supply/demand. This means you can use it in your solutions as desired.

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

6.2 Lower Bounds on Flow

Recall the following problem from a recent discussion section:

Imagine that next quarter there are N highly-demanded ICS classes all scheduled for the same time slot. There are also M students, each of which has some personal subset of those classes that they would be happy to enroll in any one of. Given:

- for each class $i \in \{1, 2, \ldots, N\}$, the max enrollment e_i
- for each student $j \in \{1, 2, \ldots, M\}$, the personal subset $P_j \subseteq \{1, 2, \ldots, N\}$ of classes they are interested in taking

In discussion, we used network flow techniques to design an algorithm that assigns every student to one of the classes in their personal subset, without exceeding the enrollment caps (or determines that this is impossible).

Question 18. Solve this problem again, but instead of a single source and sink, rephrase this as using supply and demand so we can use the algorithm we just saw.

Question 19. Now suppose we add the following constraint:

- for each class $i \in \{1, 2, \ldots, N\}$, in addition the max enrollment e_i, we also have a lower bound l_i for the number of students who must be enrolled (otherwise the class is canceled). You may assume that $l_i \leq e_i$ for all i.

Show how to use network flow techniques to design an algorithm that assigns every student to one of the classes in their personal subset, without exceeding the enrollment caps, but also providing at least the minimum number of students in each class (or determines that this is impossible).

Question 20. (Reinforcement) Generalize the approach here for any flow network with supply/demand and with at least one edge with a *lower bound* on its flow.

Upon completion of this, you now have a library function for network flow with supply/demand and lower bounds. This means you can use it in your solutions as desired.

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

7 Baseball Elimination

Suppose T is a set of teams in a sports league. At any point during the season, each time i has some number of wins, w_i and some number of games left to play g_i (for each team j, team i has $g_{i,j}$ games left against j). Our goal is to determine if team i still has a chance to finish in first place. Note that sometimes a team cannot do this, even if they win every remaining game. For purposes of this problem, games cannot end in a tie.

Let $W_k = w_k + g_k$ be the maximum number of wins possible for team k. There are two ways for team k to be eliminated:

- $W_k < w_i$ for some team i. That is, even if k wins out and i loses out, i still finishes with more wins.
- Some combination of games "mathematically eliminate" team k; that is, there is a set of teams T that will finish with a *strictly higher* average number of wins than W. Note that this means the league does not allow games to end in a tie. More formally:

$$\sum_{x \in T} w_x + \sum_{x,y \in T} g_{x,y} > W \cdot |T|$$

We are going to use network flow techniques to determine if a given team k has been eliminated. The idea is going to be to use network flow to determine if there is a way to allocate wins across every series such that no team has strictly more wins than team k. The resulting flow should allow us to either determine that team k has a possible outcome in which they finish with the most wins, or can demonstrate to an observer (who hasn't taken this class) why that cannot happen.

Let's see a few examples.

Question 21. It is near the end of the season and teams A, B, C, and D are each in contention. Team A has 92 wins, B and C have 91 wins each, and D has 90 wins. All pairs of those teams have one game remaining, *except* A and D will not play one another. Can team D finish with at least a tie for first?

Question 22. Consider the following table of games remaining. Has team D been eliminated?

Team	Wins	Games Left	Schedule ($g_{i,j}$)			
i	w_i	g_i	A	B	C	D
A	90	11		1	6	4
B	88	6	1		1	4
C	87	11	6	1		4
D	79	12	4	4	4	

Question 23. Use network flow to solve the general baseball elimination problem

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

8 Airline Scheduling

Suppose you're managing a fleet of airplanes for an airline and would like to create a flight schedule for them. You have identified a set of m flight segments that you would like to serve; each has an origin airport, a destination airport, a departure time, and an arrival time. We have a fleet of k airplanes available to us.

For example, we might have the following six possible flights:

- Boston (6AM) to Washington D.C. (7AM)
- Philadelphia (7AM) to Pittsburgh (8AM)
- Washington D.C. (8AM) to Los Angeles (11AM)
- Philadelphia (11AM) to San Francisco (2PM)
- San Francisco (2:15 PM) to Seattle (3:15PM)
- Las Vegas (5PM) to Seattle (6PM)

There are various ways an airline might determine if a plane can be reused between flights, but for this problem, let's say that flight j is either *reachable* from flight i or it isn't, and that this information is somehow available to us.

We will need to represent the following *ideas* in the graph:

- Each flight on the list must be served
- The same plane can sometimes perform two flights
- Any flight can be the first of the day for any given plane.
- Any flight can be the last of the day for any given plane.
- If we can service every flight with fewer than k planes, we are under no obligation to use all k.

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.